

a semaphore set that can contain more than one semaphore to access multiple critical sections.

The program seen in the following code snippet [Program 2] demonstrates how the semaphore set can be used for multiple critical sections using two semaphores.

```
1 /* SemMutAcc.c - This program will demonstrate synchroniz
2  * multiple critical sections using multiple semaphores */
3
4 #include <stdio.h>
5 #include <sys/sem.h>
6
7 main() {
8     int keysemid;
9     key = ftok(".", "m");
10    semid = semget(key, 2, 0);
11    struct sembuf s[2] = {{0, -1, 0}, SEM_UNDO}, {1, -1, 0}, SEM_UNDO};
12
13    printf("locking semaphore1\n");
14    semop(semid, &s[0], 1);
15    printf("Inside critical section 1 --- semaphore1\n");
16    printf("ENTER to unlock semaphore1\n");
17    getchar();
18    s[0].sem_op = 1;
19    semop(semid, &s[0], 1);
20    printf("semaphore1 unlocked\n");
21
22    printf("locking semaphore2\n");
23    semop(semid, &s[1], 1);
24    printf("Inside critical section 2 --- semaphore2\n");
25    printf("ENTER to unlock semaphore2\n");
26    getchar();
27    s[1].sem_op = 1;
28    semop(semid, &s[1], 1);
29    printf("semaphore2 unlocked\n");
30 }
```

As the semaphore set has already been created with two semaphores in Program 1, in Line 10 of Program 2 – in the *semget* system call—0 is passed as the last argument since the semaphore has already been created.

In Line 11 the structure is initialised with values.

For the first structure, 0 is passed to indicate the first semaphore, and 1 to indicate the second semaphore in the second structure. The second member indicates that the value of the semaphore is decrementing, and the third member determines what the *semop* function will do if the semaphore is busy:

- If the value is 0, it will wait until the semaphore is available.
- If the value is *IPC_NOWAIT*, it will not wait for the semaphore, but returns with an error.

Sometimes, if a process locks the semaphore and exits abnormally without unlocking, then the other processes that are waiting for the semaphore have to wait forever (in the case of *sem_flg=0*). The created semaphore set becomes unusable and the processes need to be killed

forcibly. To solve this issue, *SEM_UNDO* can be OR'ed with 0. This *SEM_UNDO* flag will automatically release the semaphore if a process exits without releasing it.

The locking and unlocking operation can be performed by the *semop* system call.

Lines 17-19 are the virtual critical section-1.

Programmers can write the critical section of their code here. The library function *getchar()* waits for the *Enter* key, and if the same executable program is being executed by another terminal, the control stops at line 15, as the critical section-1 is already locked. The second terminal user cannot see the output of Line 17. After the critical section-1 access is over, the value of the semaphore must be incremented. First the value of the semaphore is reassigned in the *sembuf* structure and then the *semop* system call is called, so that the next waiting process for the semaphore can enter the critical section after decrementing the value of the semaphore. Once the first process leaves the critical section-1, the first terminal user can see the output of Line 22, which indicates that the semaphore is released; so now the second terminal user enters the critical section and can see the output of Line 17.

Now that the binary semaphore is handled, the second semaphore, which is the counting type, should be considered. Accessing the second semaphore indicated as 1 in the *sembuf* structure, the process enters the critical section-2 after leaving the critical section-1, and the first terminal user can see the output of Line 26. The other terminal process that is accessing the 1st critical section then leaves it and enters the critical section-2. So the user in the other terminal also can see the output of Line 26. Now it can be seen that both the processes are able to enter the critical section-2, which is because a counting semaphore was created with a count of 2. If another terminal is opened and the same program is executed, and if the process tries to enter the critical section-2, then it will not be given access and at this terminal the user cannot see the output of Line 26 as it will be blocked from the second critical section.

Consider a scenario where two binary semaphores are created. This can be understood from the following code snippet [Program 3]:

```
1 /* SemDeadLockCr.c - This program creates two binary semaphores */
2
3 #include <sys/sem.h>
4 #include <stdio.h>
5
6 union semun{
7     int val;
8     struct semid_ds *buf;
9     unsigned short int *array;
10 }arg;
11
12 main() {
13     int keysemid, ret;
```